

LongBench Project Plan

1. Team Roles and Responsibilities

Ansh — Data pipeline, dataset loading, truncation, pilot inference

Shlok — Model backend, evaluation harness, metrics

Meghna — Prompting (Direct Answer and Extract-Then-Answer), inference pipeline

Divyam — Logging, span detection, latency/token tracking

Rudrani — Annotation schema, error taxonomy, inter-annotator agreement

2. Dependency Order

Dataset loading → **Truncation** → **Prompt templates** → **Model backend** → **Inference** → **Logging** → **Metrics** → **Annotation** → **Plots**

3. Execution Phases

Phase 1: Dataset preparation, prompts, backend, and logging

Phase 2: Pilot inference at 8K and 32K

Phase 3: Full experiments at 64K and 128K

Phase 4: Annotation and metrics

Phase 5: Plots, analysis, and final report

4. Detailed Implementation Plan

Here is the complete implementation plan for everything claimed as done in the report, broken down by person and file.

Project Structure

longbench-diagnostic/

├── data/

```

├── longbench_v2/      # raw JSON (downloaded)
│   ├── splits/      # truncated JSONL per budget
│   │   ├── 8k.jsonl
│   │   └── 32k.jsonl
├── src/
│   ├── dataset_loader.py  # Ansh
│   ├── truncate.py        # Ansh
│   ├── models/
│   │   ├── base.py        # Shlok
│   │   └── local_hf.py     # Shlok
│   ├── metrics.py         # Shlok
│   ├── logging_utils.py   # Divyam
│   ├── prompts/
│   │   ├── direct_answer.py # Meghna
│   │   ├── e2a_stage1.py    # Meghna
│   │   ├── e2a_stage2.py    # Meghna
│   │   ├── e2a_pipeline.py  # Meghna
│   │   └── parse_response.py # Meghna
│   ├── span_detect.py     # Divyam
│   └── iaa.py             # Rudrani
├── annotation/
│   ├── annotation_guide.md # Rudrani
│   ├── schema.json        # Rudrani
│   └── iaa_pilot.csv       # Rudrani
├── results/
│   ├── pilot_8k.jsonl      # Ansh
│   └── pilot_32k.jsonl     # Meghna
├── tests/
└── ...

```

Ansh — Data Pipeline and Pilot Inference

src/dataset_loader.py

What it does:

Loads LongBench v2 from local JSON, validates every field, and exposes clean access by category.

Functions to implement:

```
load(data_dir: str) -> list[dict]
```

Reads all JSON files under `data/longbench_v2/`. Each instance must have: `id`, `input` (the

context), `question`, `choices` (list of 4 strings), `answer` (A/B/C/D), and `category`. Returns a flat list of validated dictionaries. Raises `ValueError` on any missing field.

```
get_by_category(instances: list[dict]) -> dict[str, list[dict]]
```

Groups the flat list by `category` key. Returns a dictionary mapping category name to list of instances.

Test:

Assert `len(load(...)) == 503`. Assert all 6 expected category names are present. Assert no instance has a `None` field.

`src/truncate.py`

What it does:

Tokenizes the context field of an instance, truncates to a given token budget, decodes back to text, and returns a new instance dictionary with the truncated context. Question and choices remain unchanged.

Functions to implement:

```
truncate(instance: dict, budget: int, tokenizer) -> dict
```

Tokenizes `instance["input"]` using the provided Hugging Face tokenizer. Keeps the first `budget` tokens. Decodes back with `skip_special_tokens=True`. Returns a shallow copy of the instance with only `input` replaced. Never modifies the original.

```
generate_splits(instances: list[dict], budgets: list[int], tokenizer, out_dir: str)
```

For each budget in `budgets`, calls `truncate()` on every instance, then writes the results to `out_dir/{budget//1000}k.jsonl` as newline-delimited JSON. Skips budgets where the output file already exists.

Test:

After truncating with `budget=8192`, assert `len(tokenizer(result["input"])["input_ids"]) <= 8192`. Assert `result["question"] == instance["question"]`. Assert `result["choices"] == instance["choices"]`.

Pilot Inference Script

What it does:

Samples 50 instances stratified across categories (use `random.sample` per category with a fixed seed), runs `LocalHFBackend.predict()` on each using the Direct Answer prompt at 8K budget, and writes results to `results/pilot_8k.jsonl`. Meghna runs the same script at 32K.

Each result record must contain:

`instance_id`, `category`, `budget`, `prompt_strategy`, `model_id`, `prediction`, `ground_truth`, `correct` (bool), `input_tokens`, `output_tokens`, `latency_ms`

Shlok — Evaluation Harness

`src/models/base.py`

What it does:

Defines the abstract interface every model backend must implement.

Class to implement:

```
class LLMBackend(ABC)
```

```
predict(prompt: str) -> dict
```

Abstract method. Must return: `{"prediction": str, "input_tokens": int, "output_tokens": int, "latency_ms": float}`. `prediction` must be a single character: A, B, C, or D. If the model output is unparseable, return `{"prediction": None, ...}` — never raise.

`src/models/local_hf.py`

What it does:

Implements `LLMBackend` for Llama-3.1-8B-Instruct running locally via Hugging Face `pipeline`.

Class to implement:

```
class LocalHFBackend(LLMBackend)
```

```
__init__(self, model_id: str, device: str = "cuda")
```

Loads model and tokenizer. Sets `temperature=0` for deterministic decoding. Stores `model_id` for logging.

```
predict(self, prompt: str) -> dict
```

Records start time with `time.perf_counter()`. Runs the pipeline. Records end time. Counts `input_tokens` by tokenizing the prompt directly. Counts `output_tokens` from the generated portion only (`total - input`). Calls `parse_response.extract_choice()` on the raw output. Returns the standard dictionary.

Test:

On a toy prompt with a clearly stated answer, assert `prediction` is one of `{A, B, C, D, None}`. Assert `latency_ms > 0`. Assert `input_tokens > 0`.

`src/metrics.py`

What it does:

All accuracy computation and confidence intervals. This is authoritative — no one else computes metrics independently.

Functions to implement:

```
accuracy(results: list[dict]) -> float
```

`correct` is a boolean field. Returns `sum(correct) / len(results)`. Raises if the list is empty.

```
per_category_accuracy(results: list[dict]) -> dict[str, float]
```

Groups by `category`, calls `accuracy()` on each group. Returns `{category: acc}`.

```
bootstrap_ci(results: list[dict], n_resamples: int = 10000, ci: float = 0.95) -> tuple[float, float]
```

Resamples `results` with replacement `n_resamples` times. Computes `accuracy()` on each resample. Returns `(lower, upper)` percentile bounds for the requested confidence interval. Use `numpy` for vectorized resampling; do not loop over resamples in Python.

Test:

On a results list of 100 items all correct, assert `accuracy == 1.0` and CI is `(1.0, 1.0)`. On a 50/50 list, assert the CI contains `0.5`.

Meghna — Prompt Framework and E2A Pipeline

src/prompts/parse_response.py

What it does:

Parses raw model output into a single answer letter. Used by all backends.

Function to implement:

```
extract_choice(raw: str) -> str | None
```

Strips whitespace. Looks for a standalone A/B/C/D using `re.search(r'\b([ABCD])\b', raw)` on the uppercased string. If exactly one unique match, returns it. If zero or multiple distinct matches, returns `None`. Never raises.

Test:

```
extract_choice("The answer is B.") → "B"
```

```
extract_choice("A or B") → None
```

```
extract_choice("") → None
```

```
extract_choice("(C)") → "C"
```

src/prompts/direct_answer.py

What it does:

Builds the Direct Answer prompt string from an instance dictionary.

Function to implement:

```
build_da_prompt(instance: dict) -> str
```

Constructs the prompt in this exact structure:

Read the following context carefully and answer the multiple choice question.

Context:

```
{instance["input"]}
```

Question: {instance["question"]}

A. {instance["choices"][0]}

B. {instance["choices"][1]}

C. {instance["choices"][2]}

D. {instance["choices"][3]}

Answer with a single letter: A, B, C, or D.

Returns the string. Does not call the model.

Test:

Assert the returned string contains "Answer with a single letter". Assert all four choice letters appear. Assert `instance["input"]` appears verbatim.

`src/prompts/e2a_stage1.py`

What it does:

Builds the extraction prompt for stage 1 of E2A — asks the model to pull out relevant evidence before answering.

Function to implement:

```
build_extraction_prompt(instance: dict) -> str
```

Constructs:

Read the following context carefully.

Context:

{instance["input"]}

Question: {instance["question"]}

Extract the most relevant sentences or passages from the context that would help answer this question. Copy them verbatim. If nothing is relevant, write "No relevant passage found."

`src/prompts/e2a_stage2.py`

What it does:

Builds the answer prompt for stage 2, using only the extracted evidence.

Function to implement:

```
build_answer_prompt(instance: dict, extracted_evidence: str) -> str
```

Constructs:

Using only the extracted evidence below, answer the multiple choice question.

Extracted evidence:

{extracted_evidence}

Question: {instance["question"]}

A. {instance["choices"][0]}

- B. {instance["choices"]}[1]}
- C. {instance["choices"]}[2]}
- D. {instance["choices"]}[3]}

Answer with a single letter: A, B, C, or D.

src/prompts/e2a_pipeline.py

What it does:

Orchestrates both E2A stages using any `LLMBackend`.

Function to implement:

```
run_e2a(instance: dict, model: LLMBackend) -> dict
```

Calls `build_extraction_prompt()`, then `model.predict()` to get the evidence using the raw output, not `extract_choice()`. Calls `build_answer_prompt()` with the raw evidence string, then `model.predict()` again to get the final answer. Returns:

```
{
    "instance_id": instance["id"],
    "extracted_evidence": str,
    "prediction": str | None,
    "ground_truth": instance["answer"],
    "correct": bool,
    "input_tokens": int,
    "output_tokens": int,
    "latency_ms": float
}
```

Test:

Mock `model.predict()` with fixed outputs. Assert `input_tokens` equals the sum of both calls. Assert `correct` is computed as `prediction == ground_truth`.

Divyam — Logging and Span Detection

src/logging_utils.py

What it does:

Structured per-call logging appended to JSONL result files. Integrated into both the Direct Answer script and `run_e2a()`.

Class to implement:

```
class InferenceLogger
```

```
__init__(self, out_path: str)
```

Opens `out_path` in append mode. Stores file handle.

```
log(self, record: dict)
```

Validates that `record` contains all required keys: `instance_id`, `category`, `budget`, `prompt_strategy`, `model_id`, `prediction`, `ground_truth`, `correct`, `input_tokens`, `output_tokens`, `latency_ms`. Raises `KeyError` if any are missing. Writes `json.dumps(record) + "\n"` to file.

```
close(self)
```

Flushes and closes the file handle.

Use as a context manager by implementing `__enter__` and `__exit__` so it can be used with `with InferenceLogger(...) as logger`.

Test:

Write 3 records, read back the file, and assert 3 valid JSON lines. Assert `KeyError` is raised on a record missing `latency_ms`.

src/span_detect.py

What it does:

Identifies the answer-relevant sentence in a single-document QA instance using BM25, for use in the needle-position perturbation experiment.

Functions to implement:

```
segment_sentences(text: str) -> list[str]
```

Uses `nltk.sent_tokenize()`. Filters out sentences shorter than 10 characters. Returns a list of sentence strings.

```
find_relevant_span(instance: dict, threshold: float = 0.5) -> dict
```

Segments `instance["input"]` into sentences. Builds a BM25 index over those sentences using `rank_bm25.BM25Okapi`. Tokenizes `instance["question"]` as the query (split on whitespace, lowercased). Retrieves the top-1 sentence and its normalized BM25 score (`score / max score in the corpus`). Returns:

```
{
  "span_text": str,
  "sentence_index": int,
  "score": float,
  "flagged": bool
}
```

If `instance["category"]` is not `"single-document QA"`, raises `ValueError` because this function is only valid for single-document tasks.

Test:

On a synthetic instance where the question keywords appear in exactly one sentence, assert `flagged == False` and `span_text` contains those keywords. Assert `ValueError` is raised for a multi-document category instance.

Rudrani — Annotation Schema and IAA

annotation/schema.json

What it contains: Machine-readable label specification.

```
{
  "primary_categories": [
    {
      "id": "RF",
      "label": "Retrieval Failure",
      "definition": "Model selects an answer not supported by any passage in the context.",
      "decision_cue": "Could a careful human reader find evidence for the predicted answer in the context? If no, RF."
    },
    {
      "id": "RsF",
      "label": "Reasoning Failure",
      "definition": "Relevant passage is present and identifiable, but the model draws an incorrect inference.",
      "decision_cue": "Is the correct evidence present? If yes, and the model still got it wrong, RsF."
    },
    {
      "id": "INC",
      "label": "Instruction Non-Compliance",
      "definition": "Answer format or task constraint violated despite sufficient information."
    }
  ]
}
```

```
    "decision_cue": "Did the model output something other than A/B/C/D, or violate an explicit constraint?"
  },
],
"secondary_attributes": [
  {
    "id": "HALL",
    "label": "Hallucination",
    "definition": "Answer relies on information not present anywhere in the provided context.",
    "type": "binary"
  }
]
}
```

annotation/annotation_guide.md

What it contains:

The human-readable guide annotators use. It must include:

- A decision flowchart as a numbered list:
 1. Is output A/B/C/D? If no → INC.
 2. Is there any passage in the context that supports the predicted answer? If no → RF.
 3. Is there a passage that supports the correct answer? If yes → RsF.
 - 3 worked examples per primary category: the instance question, the model's wrong prediction, the ground truth, and the label with a one-sentence justification.
 - 3 counter-examples: cases that look like one category but are actually another, with explanation.
 - Hallucination flag instructions: set HALL=1 if the model's prediction references a specific fact not present anywhere in the context, regardless of primary label.
 - Adjudication protocol: any disagreement gets a 2-sentence written resolution added to a shared [adjudication_log.md](#).
-

src/iaa.py

What it does:

Computes inter-annotator agreement for the 20-instance pilot.

Function to implement:

```
cohen_kappa(labels_a: list[str], labels_b: list[str]) -> float
```

Validates both lists have equal length and contain only valid category IDs (RF, RsF, INC).

Computes Cohen's κ using `sklearn.metrics.cohen_kappa_score`. Returns the float.

Raises `ValueError` if the lists contain invalid labels.

```
summarize_disagreements(labels_a: list[str], labels_b: list[str],  
instance_ids: list[str]) -> list[dict]
```

Returns a list of `{"instance_id": str, "annotator_a": str, "annotator_b": str}` for every position where the labels differ.

Test:

On two identical label lists, assert $\kappa = 1.0$. On completely random disagreement, assert $\kappa < 0.5$.

Assert `ValueError` on invalid label like "XYZ".

annotation/iaa_pilot.csv

What it contains:

20 rows drawn from incorrect predictions in `results/pilot_8k.jsonl`.

Columns:

```
instance_id, category, budget, model_prediction, ground_truth,  
annotator_a_label, annotator_a_hall, annotator_b_label, annotator_b_hall,  
resolved_label, resolution_note
```

`resolved_label` and `resolution_note` are filled in after the adjudication session.

`annotator_a_hall` and `annotator_b_hall` are 0 or 1.

Shared Result Format

Every JSONL result file — whether from Direct Answer or E2A, pilot or full sweep — must use this exact schema so all downstream code can read any file without branching:

```
{  
  "instance_id": "string",  
  "category": "string",  
  "budget": 8192,  
  "prompt_strategy": "DA",  
  "model_id": "meta-llama/Llama-3.1-8B-Instruct",  
}
```

```
"prediction": "B",  
"ground_truth": "A",  
"correct": false,  
"input_tokens": 7841,  
"output_tokens": 3,  
"latency_ms": 4210.3  
}
```

This schema is the integration contract between Ansh and Meghna for inference, Shlok for metrics, Divyam for logging and plots, and Rudrani for annotation sampling. No one deviates from it without team agreement.